SOC CYBER THREAT INVESTIGATION PLATFORM

Software Requirements Specification

Version 1.0 | April 2026

Document Title	SRS — SOC Cyber Threat Investigation Platform
Version	1.0
Status	Final
Date	April 2026
Project Type	Cybersecurity Internship / Academic Project
Scope	5-Phase SOC Tooling Platform (Python + Flask)
Author	SOC Development Team
Reviewer	Project Supervisor

Table of Contents

1. Introduction
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definitions & Acronyms
   - 1.4 References
2. Overall Description
   - 2.1 Product Perspective
   - 2.2 Product Functions
   - 2.3 User Classes
   - 2.4 Assumptions & Dependencies
3. System Architecture
   - 3.1 Architecture Overview
   - 3.2 Module Breakdown
   - 3.3 Technology Stack
4. Functional Requirements
   - 4.1 Log Analyzer
   - 4.2 Network Scanner
   - 4.3 Packet Analyzer
   - 4.4 Threat Intelligence
   - 4.5 Web Application Dashboard
5. Non-Functional Requirements
   - 5.1 Performance
   - 5.2 Security
   - 5.3 Usability
   - 5.4 Reliability
6. Use Cases
   - UC-01 through UC-05
7. External Interface Requirements
   - 7.1 User Interfaces
   - 7.2 API Interfaces
   - 7.3 Hardware Interfaces
8. Data Requirements
   - 8.1 Data Models
   - 8.2 Database Schema
9. Constraints & Compliance
10. Appendix
    - A. Glossary
    - B. Attack Signature Library

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the SOC Cyber Threat Investigation Platform — a five-phase cybersecurity tooling suite developed as part of a Security Operations Centre (SOC) internship project. The document serves as the authoritative reference for design, development, testing, and review.

1.2 Scope

The platform consists of five integrated modules delivered across five development phases:

Phase 1	**Log Analyzer**	Parse auth.log / syslog files to detect brute-force attacks and unauthorised access.
Phase 2	**Network Scanner**	Perform TCP port scanning, banner grabbing, and ICMP ping sweep on target hosts.
Phase 3	**Packet Analyzer**	Inspect live or captured network packets for SQL injection, XSS, reverse shells, and obfuscated payloads.
Phase 4	**Threat Intelligence**	Scrape and store public malicious IP feeds; correlate observed IPs against threat data.
Phase 5	**Web Application**	Flask-based unified dashboard integrating all four modules with REST API and report export.

1.3 Definitions and Acronyms

`SOC`	Security Operations Centre — team responsible for monitoring and responding to threats.
`SRS`	Software Requirements Specification.
`SQLi`	SQL Injection — attack embedding SQL commands in user input.
`XSS`	Cross-Site Scripting — injection of malicious client-side scripts.
`RCE`	Remote Code Execution — ability of an attacker to run arbitrary code on a target.
`PCAP`	Packet Capture — binary file storing raw network packet data.
`C2`	Command & Control — server used by attackers to communicate with compromised hosts.
`IoC`	Indicator of Compromise — forensic artefact indicating a breach.
`TTPs`	Tactics, Techniques, and Procedures — attacker behaviour patterns.
`CVSS`	Common Vulnerability Scoring System — standardised severity rating.

1.4 References

- OWASP Top 10 Web Application Security Risks (2021)
- MITRE ATT&CK; Framework v14 — https://attack.mitre.org
- Feodo Tracker Blocklist — https://feodotracker.abuse.ch

- Emerging Threats Open Ruleset — https://rules.emergingthreats.net
- Scapy Documentation — https://scapy.readthedocs.io
- Flask Documentation — https://flask.palletsprojects.com
- NIST SP 800-61 Rev.2 — Computer Security Incident Handling Guide

2. Overall Description

2.1 Product Perspective

The platform is a standalone desktop/server application that replicates core SOC analyst workflows in a single Python + Flask codebase. It does not replace commercial SIEM tools but provides a learning-oriented, fully open implementation of the same detection pipelines used in real-world Security Operations Centres.

2.2 Product Functions

- Parse and analyse system authentication logs to identify brute-force and credential-stuffing attacks.
- Perform multi-threaded TCP port scanning with service banner grabbing and risk classification.
- Capture and inspect network packets in real time or from .pcap files for injection and malware payloads.
- Aggregate threat intelligence from six public blocklist feeds into a local SQLite database.
- Correlate observed IP addresses from log files against the threat intelligence database.
- Provide a unified web dashboard with drag-and-drop file upload, live scan input, and JSON report export.
- Generate structured incident reports containing findings from all active modules.

2.3 User Classes and Characteristics

SOC Analyst (Primary)	Monitors alerts, investigates IPs, reviews scan results. Technical user — familiar with networking and Linux.
Security Intern	Uses the platform for hands-on learning of SOC tooling workflows. May require guided operation.
Incident Responder	Consumes exported JSON/PDF reports during active incident response. Read-only consumer.
Platform Administrator	Configures feed update schedules, manages the SQLite database, deploys the Flask server.

2.4 Assumptions and Dependencies

- Python 3.10 or higher is installed on the deployment host.
- The operator has legal authorisation to scan any network targets used with the Network Scanner.
- Live packet capture (Phase 3) requires root or administrator privileges on the host OS.
- Threat feed updates (Phase 4) require outbound HTTPS access to public blocklist providers.
- The system is deployed in an isolated lab or controlled network environment for security.
- Flask runs in development mode; production deployment should add Gunicorn + NGINX.

3. System Architecture

3.1 Architecture Overview

The platform follows a modular monolith architecture. Each of the four detection engines is implemented as an independent Python module with no coupling between them. The Flask web application (Phase 5) acts as an orchestration layer, importing each module and exposing its functionality via REST API endpoints. All persistent data is stored in SQLite.

3.2 Module Breakdown

Layer	Component	Responsibility	File
Presentation	**Dashboard UI**	Dark terminal-themed HTML/CSS/JS single-page app	`templates/index.html`
API	**Flask Backend**	REST routes, file upload handling, report export	`app.py`
Detection	**Log Analyzer**	Regex-based brute-force and auth event parsing	`modules/log_analyzer.py`
Detection	**Network Scanner**	Threaded TCP connect scan + banner grabbing	`modules/network_scanner.py`
Detection	**Packet Analyzer**	Scapy packet inspection + signature matching	`modules/packet_analyzer.py`
Intelligence	**Threat Intel**	Feed scraping, SQLite storage, IP correlation	`modules/threat_intel.py`
Storage	**SQLite Database**	Threat IP store, feed metadata, correlation log	`threat_intel.db`
Storage	**File System**	Uploaded logs/pcaps, JSON report archive	`uploads/` `reports/`

3.3 Technology Stack

Language: Python 3.10+

Web Framework: Flask 3.x

Packet Library: Scapy 2.5+

HTTP Client: requests + BeautifulSoup4

Database: SQLite 3 (via stdlib sqlite3)

Frontend: Vanilla HTML5 / CSS3 / JavaScript (no framework)

Concurrency: concurrent.futures.ThreadPoolExecutor

Report Export: JSON (stdlib) + ReportLab (PDF)

Containerisation: Not required — pure Python, runs on Linux / macOS / Windows

4. Functional Requirements

4.1 Log Analyzer (Phase 1)

Req. ID	Requirement Description	Priority
FR-L01	The system shall parse standard Linux auth.log and syslog files line by line using compiled regular expressions.	HIGH
FR-L02	The system shall extract timestamp, IP address, username, and event type (failure, success, invalid user) from each log line.	HIGH
FR-L03	The system shall detect brute-force attacks by counting failed authentication attempts per source IP within a configurable sliding time window (default: 5 failures in 60 seconds).	CRITICAL
FR-L04	The system shall flag BRUTE_FORCE_SUCCESS events when a successful login follows 3 or more failures from the same IP within 5 minutes.	CRITICAL
FR-L05	The system shall detect username enumeration by flagging IPs with 3 or more 'Invalid user' log entries.	HIGH
FR-L06	The system shall classify each alert with severity levels: CRITICAL, HIGH, MEDIUM, or LOW.	HIGH
FR-L07	The system shall export all findings to a structured JSON report file when requested.	MEDIUM
FR-L08	The system shall support log files up to 500,000 lines without exceeding 60 seconds processing time.	MEDIUM

4.2 Network Scanner (Phase 2)

Req. ID	Requirement Description	Priority
FR-N01	The system shall perform TCP SYN/connect scans against a list of at least 24 common ports per target host.	HIGH
FR-N02	The system shall support scanning a user-defined port range (e.g. 1–65535) in addition to the default top-port list.	MEDIUM
FR-N03	The system shall use thread-pool concurrency (minimum 100 threads) to complete top-port scans in under 3 seconds per host.	HIGH
FR-N04	The system shall perform service banner grabbing on all open ports to identify application and version strings.	HIGH
FR-N05	The system shall classify every detected open port against a risk database with severity (CRITICAL/HIGH/MEDIUM/LOW) and a human-readable reason.	CRITICAL
FR-N06	The system shall support ICMP ping sweep of a /24 subnet to identify live hosts, with TCP fallback when raw sockets are unavailable.	MEDIUM

FR-N07	The system shall mark the following ports as CRITICAL risk: 23 (Telnet), 445 (SMB), 3306 (MySQL), 5432 (PostgreSQL), 6379 (Redis), 8888 (Jupyter), 27017 (MongoDB).	**CRITICAL**

4.3 Packet Analyzer (Phase 3)

Req. ID	Requirement Description	Priority
FR-P01	The system shall read and inspect packets from .pcap and .pcapng files produced by Wireshark or tcpdump.	HIGH
FR-P02	The system shall support live packet capture on a specified network interface (requires root/admin).	MEDIUM
FR-P03	The system shall detect SQL injection patterns including UNION SELECT, OR 1=1, DROP TABLE, time-based blind injection (sleep/waitfor), and stacked queries.	CRITICAL
FR-P04	The system shall detect Cross-Site Scripting (XSS) patterns including script tags, event handler attributes, and javascript: URI schemes.	HIGH
FR-P05	The system shall detect OS command injection via semicolon/pipe/backtick chaining and system path access (/etc/passwd, /bin/sh).	CRITICAL
FR-P06	The system shall detect reverse shell payloads including bash /dev/tcp, netcat -e, Python socket, Perl socket, and PHP fsockopen patterns.	CRITICAL
FR-P07	The system shall detect directory traversal sequences including ../ chains and URL-encoded variants (%2e%2e%2f).	HIGH
FR-P08	The system shall decode URL-encoded (single and double) and base64-encoded payloads before signature matching to detect obfuscated attacks.	HIGH
FR-P09	The system shall operate in demo mode using built-in synthetic payloads when Scapy is not installed.	LOW

4.4 Threat Intelligence (Phase 4)

Req. ID	Requirement Description	Priority
FR-T01	The system shall scrape and parse IP blocklists from at least 6 public threat intelligence feeds including Feodo Tracker, Blocklist.de (SSH + All), Emerging Threats, CINS Army, and TorDNSEL.	HIGH
FR-T02	The system shall store all collected threat IPs in a local SQLite database with category, severity, source feed, first-seen, last-seen, and hit-count fields.	HIGH
FR-T03	The system shall deduplicate IPs using ON CONFLICT UPSERT logic, incrementing a hit_count on repeated observations.	MEDIUM
FR-T04	The system shall look up any single IP or batch of IPs against the local database and return match status, category, severity, and feed source.	CRITICAL
FR-T05	The system shall correlate all IPs extracted from an uploaded log file against the threat database and record each match in a correlation_log table.	CRITICAL
FR-T06	The system shall enrich matched IPs with geolocation data (country, city, ISP/ASN) using the ip-api.com free API.	MEDIUM
FR-T07	The system shall cache geolocation responses in memory to avoid duplicate API calls within a session.	LOW

FR-T08	The system shall operate in offline/demo mode using a built-in simulated feed dataset when network access is unavailable.	**MEDIUM**

4.5 Web Application Dashboard (Phase 5)

Req. ID	Requirement Description	Priority
FR-W01	The system shall provide a single-page web dashboard accessible at http://localhost:5000 with navigation to all five modules.	HIGH
FR-W02	The dashboard shall display an overview panel showing aggregate alert counts from all completed module runs.	HIGH
FR-W03	The Log Analyzer tab shall accept drag-and-drop or file-picker upload of auth.log files and display parsed results within 5 seconds.	HIGH
FR-W04	The Network Scanner tab shall accept a target IP or hostname input and display scan results with per-port risk classification.	HIGH
FR-W05	The Packet Analyzer tab shall accept .pcap file upload and provide a built-in demo mode without requiring a real capture file.	HIGH
FR-W06	The Threat Intelligence tab shall accept multi-line IP input and display reputation results with geolocation and feed attribution.	HIGH
FR-W07	The Incident Report tab shall auto-generate a consolidated report from all completed module runs in the current session.	HIGH
FR-W08	The system shall expose REST API endpoints for each module accepting JSON or multipart/form-data and returning JSON responses.	HIGH
FR-W09	All uploaded files and generated reports shall be saved to the uploads/ and reports/ directories respectively.	MEDIUM
FR-W10	The system shall support export of the full incident report as a downloadable JSON file.	MEDIUM

5. Non-Functional Requirements

5.1 Performance

Req. ID	Requirement Description	Priority
NFR-P01	The Log Analyzer shall process a 10,000-line auth.log file in under 2 seconds on a standard laptop (i5, 8 GB RAM).	HIGH
NFR-P02	The Network Scanner shall complete a top-24-port scan of a single host in under 3 seconds using 100+ concurrent threads.	HIGH
NFR-P03	The Packet Analyzer shall process 1,000 packets per second minimum when reading from a .pcap file.	MEDIUM
NFR-P04	The Threat Intelligence feed update shall process and store 10,000 IPs in under 30 seconds.	MEDIUM
NFR-P05	The Flask web dashboard API shall respond to all requests within 500 ms for non-scanning operations.	HIGH

5.2 Security

Req. ID	Requirement Description	Priority
NFR-S01	The system shall sanitise all user-supplied filenames using werkzeug.utils.secure_filename before writing to disk.	CRITICAL
NFR-S02	The system shall validate target IP/hostname inputs using regex before initiating any network operations.	CRITICAL
NFR-S03	The system shall enforce a maximum upload file size of 32 MB to prevent denial-of-service via large file upload.	HIGH
NFR-S04	The system shall not store raw packet payloads containing PII in the database — only matched signatures and metadata.	HIGH
NFR-S05	The system shall operate exclusively on systems and networks for which the operator has explicit written authorisation.	CRITICAL

5.3 Usability

Req. ID	Requirement Description	Priority
NFR-U01	The web dashboard shall operate correctly on any modern browser (Chrome 100+, Firefox 100+, Edge 100+) without plugins.	HIGH
NFR-U02	All severity levels shall be colour-coded consistently: CRITICAL=red, HIGH=amber, MEDIUM=blue, LOW=green.	MEDIUM
NFR-U03	Each module shall include a demo/dry-run mode accessible with a single button click requiring no file upload.	MEDIUM

NFR-U04	The CLI for each module shall print a usage help message when invoked with no arguments.	**LOW**

5.4 Reliability

Req. ID	Requirement Description	Priority
NFR-R01	The system shall handle malformed or truncated log/pcap files gracefully without crashing — logging an error and continuing.	**HIGH**
NFR-R02	The system shall continue operating if one or more threat feed URLs are unavailable, skipping failed feeds and logging the error.	**HIGH**
NFR-R03	All database write operations shall use transactions to prevent partial writes on unexpected termination.	**HIGH**

6. Use Cases

Use Case ID	UC-01
Name	Investigate Brute-Force Attack from Log File
Actor	SOC Analyst
Precondition	auth.log file is available. System is running.
Main Flow	1. Analyst navigates to Log Analyzer tab. 2. Uploads auth.log via drag-and-drop. 3. System parses file and applies brute-force detection rules. 4. Dashboard displays alerts grouped by severity with IP, attempt count, and time window. 5. Analyst clicks an IP to expand detail and notes BRUTE_FORCE_SUCCESS for 192.168.1.105. 6. Analyst proceeds to Threat Intel tab to check the IP reputation.
Postcondition	Alerts are rendered in the dashboard. Results are saved to reports/ as JSON.

Use Case ID	UC-02
Name	Scan a Suspicious Internal Host
Actor	SOC Analyst
Precondition	Analyst has identified a suspicious IP from log analysis. Written authorisation exists.
Main Flow	1. Analyst navigates to Network Scanner tab. 2. Enters target IP address and clicks Scan. 3. System performs TCP connect scan on top 24 ports with banner grabbing. 4. Results display within 3 seconds, showing open ports colour-coded by risk. 5. Analyst notes port 3306 (MySQL) is CRITICAL — exposed database. 6. Analyst exports results and files an incident ticket.
Postcondition	Scan results displayed and saved. Analyst escalates critical findings.

Use Case ID	UC-03
Name	Analyse a Captured Network Session for Attacks
Actor	SOC Analyst / Intern
Precondition	Wireshark .pcap capture file is available from a suspicious network session.

Main Flow	1. Analyst navigates to Packet Analyzer tab. 2. Uploads .pcap file. 3. System processes all packets, decoding URL and base64 layers before signature matching. 4. Dashboard shows findings filtered by category (SQLi, XSS, ReverseShell, etc.). 5. Analyst expands the ReverseShell finding and sees bash /dev/tcp payload. 6. Analyst notes attacker IP and cross-references with Threat Intel module.
Postcondition	Packet findings logged. Attack IPs identified for further correlation.

Use Case ID	UC-04
Name	Update Threat Feeds and Correlate Log IPs
Actor	Platform Administrator
Precondition	Outbound HTTPS to blocklist providers is available.
Main Flow	1. Admin runs: python threat_intel.py --update 2. System fetches 6 live blocklists, parsing and upserting all IPs into SQLite. 3. Admin then runs: python threat_intel.py --correlate /var/log/auth.log 4. System extracts all IPs from the log, looks each up in the database. 5. Matches are printed with severity and category, and saved to correlation_log table. 6. Admin exports the results: python threat_intel.py --report --export
Postcondition	Database updated. Correlation results saved to threat_report.json.

Use Case ID	UC-05
Name	Generate and Download Incident Report
Actor	Incident Responder
Precondition	At least one module has been executed in the current dashboard session.
Main Flow	1. Responder navigates to Incident Report tab. 2. Dashboard auto-generates a consolidated report from all session findings. 3. Responder reviews the report: log alerts, open ports, packet threats, and threat intel matches. 4. Responder clicks Download JSON report. 5. Browser downloads soc_report_.json for use in the ticketing system.
Postcondition	JSON report downloaded and filed in the incident ticketing system.

7. External Interface Requirements

7.1 User Interfaces

The primary user interface is a dark-themed single-page web application served by Flask at http://localhost:5000. Navigation is provided by a persistent left sidebar. Each module occupies a full-width content panel. All modules also expose a CLI interface for headless/scripted operation.

7.2 REST API Endpoints

Method	Endpoint	Input	Response
GET	/api/log/demo	—	JSON — demo log analysis results
POST	/api/log/analyze	multipart: file (auth.log)	JSON — alerts, stats
POST	/api/scan	JSON: {target: str}	JSON — open ports, risk ratings
GET	/api/packet/demo	—	JSON — demo packet findings
POST	/api/packet/analyze	multipart: file (.pcap)	JSON — threat findings
POST	/api/threat/check	JSON: {ips: [str]}	JSON — reputation results
POST	/api/report/export	JSON: full session state	JSON — {file, message}
GET	/reports/	URL param: filename	File download (JSON)

7.3 Hardware Interfaces

The Network Scanner requires a standard TCP/IP network interface. The Packet Analyzer requires a raw socket–capable network interface for live capture (ICMP ping and live sniff). No specialised hardware is required beyond a standard workstation or server.

8. Data Requirements

8.1 Data Models

The system uses three primary data entities:

ThreatIP: Stores each unique IP observed in threat feeds with full enrichment metadata.

FeedMeta: Tracks the last update time and IP count for each configured blocklist feed.

CorrelationLog: Records every IP from a log file that matched an entry in the ThreatIP table.

8.2 Database Schema

Table: threat_ips

Column	Type	Constraint	Description
`ip`	TEXT	PK (composite)	IPv4 address string
`source`	TEXT	PK (composite)	Originating feed name
`category`	TEXT	NOT NULL	Threat category (Botnet C2, Brute-force, …)
`severity`	TEXT	NOT NULL	CRITICAL / HIGH / MEDIUM / LOW
`description`	TEXT		Human-readable threat description
`first_seen`	TEXT		ISO-8601 timestamp of first insertion
`last_seen`	TEXT		ISO-8601 timestamp of last update
`hit_count`	INTEGER	DEFAULT 1	Number of times seen across feed refreshes

Table: feed_meta

Column	Type	Constraint	Description
`feed_name`	TEXT	PK	Unique feed identifier
`last_update`	TEXT		ISO-8601 timestamp of last successful fetch
`ip_count`	INTEGER		Number of IPs fetched in last update

Table: correlation_log

Column	Type	Constraint	Description
`id`	INTEGER	PK AUTOINCREMENT	Unique row ID
`ip`	TEXT	NOT NULL	Matched IP address
`log_file`	TEXT		Path of the log file analysed
`matched_at`	TEXT		ISO-8601 match timestamp
`severity`	TEXT		Severity from threat_ips
`category`	TEXT		Category from threat_ips

9. Constraints and Compliance

Legal and Ethical Constraints

- The Network Scanner and Packet Analyzer modules must only be used against systems and networks for which the operator has explicit written authorisation. Unauthorised scanning may violate the Computer Fraud and Abuse Act (CFAA), UK Computer Misuse Act, and equivalent legislation in other jurisdictions.
- Threat intelligence data collected from public blocklists must be used for defensive purposes only, in accordance with each feed provider's terms of service.
- The system does not collect, store, or transmit any Personally Identifiable Information (PII) beyond IP addresses that appear in user-supplied log files.

Technical Constraints

- The platform is a single-user, local-deployment application. It is not designed for multi-tenant or public internet exposure.
- Flask's built-in development server is used for local operation. Production deployment requires a WSGI server (e.g. Gunicorn) and reverse proxy (e.g. Nginx).
- File uploads are limited to 32 MB. Larger pcap files should be pre-filtered with tcpdump before upload.
- The SQLite database does not support concurrent write access from multiple processes. A single Flask worker must be used.
- Live packet capture requires raw socket privileges (root on Linux, Administrator on Windows). The system gracefully falls back to file-based analysis without elevated privileges.

10. Appendix

A. Glossary of Attack Types

Brute-Force Attack — Automated repeated login attempts using a dictionary or all possible passwords.

Credential Stuffing — Using username/password pairs leaked from previous breaches to attempt login.

SQL Injection (SQLi) — Inserting SQL commands into application input fields to manipulate the database.

Cross-Site Scripting — Injecting JavaScript into a web page served to other users to steal sessions or execute code.

Command Injection — Inserting OS shell commands into application parameters that are passed to a system shell.

Directory Traversal — Using ../ sequences to escape the web root and access files outside the intended directory.

Reverse Shell — Payload that causes the victim host to connect back to the attacker's machine, giving a remote shell.

DNS Tunneling — Encoding data in DNS query subdomains to exfiltrate data or establish a C2 channel.

Port Scan — Systematically probing ports on a host to discover running services as a reconnaissance step.

C2 (Command & Control) — Infrastructure used by malware to receive instructions and exfiltrate data from compromised hosts.

B. Attack Signature Library Reference

The following table summarises the attack signature categories implemented in the Packet Analyzer (Phase 3). All patterns are compiled as Python re objects with re.IGNORECASE.

Category	Severity	Example Pattern	What it detects
SQLi	**CRITICAL**	`UNION.*SELECT | DROP TABLE | OR 1=1`	Union-based and boolean injection
SQLi	**HIGH**	`-- comment | information_schema`	Comment injection, enumeration
XSS	**HIGH**	`script tag | javascript: | onerror=`	Reflected and stored XSS
CMDi	**CRITICAL**	`; cat | ; whoami | backtick id`	Semicolon/backtick OS injection
PathTraversal	**HIGH**	`../../../ | /etc/passwd`	Directory escape, system file read
ReverseShell	**CRITICAL**	`bash /dev/tcp | nc -e shell`	Bash/netcat reverse shells
Obfuscation	**HIGH**	`base64_decode | fromCharCode`	Encoded payload execution

End of Document — SOC Cyber Threat Investigation Platform SRS v1.0 — April 2026